ODOO DEVELOPER PROGRAM

Advanced Views & Inheritance

Chapter Four – Odoo 17 Development Tutorials

Ahmed Muhumed

July 25, 2026

Outline

Chapter Four Overview

What This Chapter Covers

- Transient models and wizard forms
- Abstract models
- Python and SQL constraints
- Extension and delegation inheritance
- XPath and view extension
- Smart / stat buttons
- Advanced views: Pivot, Kanban, Calendar, Graph, Activity, Gantt, and more

Why These Topics Matter

- Real Odoo apps are not only form + tree views.
- Wizards guide users through business processes.
- Inheritance lets you extend Odoo without rewriting core modules.
- Advanced views turn records into dashboards, boards, and schedules.
- Constraints protect data quality at Python and SQL levels.

Learning Goals

- Build a wizard with a transient model and form view.
- Create reusable abstract models.
- Add Python and SQL constraints correctly.
- Extend models with `_inherit` and delegation (`_inherits`).
- Extend views with XPath inheritances.
- Add smart buttons and advanced view types.

Transient Models and Wizard Forms

What Is a Transient Model?

- A **transient model** stores temporary wizard data.
- It inherits from **`models.TransientModel`**.
- Records are cleaned automatically after some time.

Method Syntax

```
class StudentWizard(models.TransientModel):
    _name = 'student.wizard'
    _description = 'Student Wizard'
```

TransientModel vs Model

- `models.Model`: normal business data (students, invoices, partners).
- `models.TransientModel`: temporary UI/process data.

```python
# Permanent
class Student(models.Model):
    _name = 'student.student'

# Temporary wizard
class StudentWizard(models.TransientModel):
    _name = 'student.wizard'
```

- Do not store important long-term business data in transient models.

Building a Wizard Model

```python
class StudentAssignWizard(models.TransientModel):
    _name = 'student.assign.wizard'
    _description = 'Assign Classroom Wizard'

    classroom_id = fields.Many2one('school.classroom', required=True)
    student_ids = fields.Many2many('student.student', string='Students')

    def action_assign(self):
        self.ensure_one()
        self.student_ids.write({'classroom_id': self.classroom_id.id})
        return {'type': 'ir.actions.act_window_close'}
```

Wizard Form View

```
<record id="view_student_assign_wizard" model="ir.ui.view">
  <field name="name">student.assign.wizard.form</field>
  <field name="model">student.assign.wizard</field>
  <field name="arch" type="xml">
  <form string="Assign Classroom">
    <group>
    <field name="classroom_id"/>
    <field name="student_ids" widget="many2many_tags"/>
    </group>
    <footer>
    <button name="action_assign" type="object" string="Assign" class="btn-primary"/>
    <button string="Cancel" special="cancel"/>
    </footer>
  </form>
  </field>
</record>
```

Opening a Wizard with an Action

```
def action_open_assign_wizard(self):
  return {
    'type': 'ir.actions.act_window',
    'name': 'Assign Classroom',
    'res_model': 'student.assign.wizard',
    'view_mode': 'form',
    'target': 'new',
    'context': {
      'default_student_ids': [(6, 0, self.ids)],
    },
  }
```

- `target='new'` opens the wizard as a modal dialog.
- Context can prefill wizard fields.

Wizard Best Practices

- Keep wizards focused on one job.
- Validate inputs in the wizard method.
- Close the wizard with `act_window_close` or return another action.
- Use `default_*` context keys to prefill values.
- Remember: transient data is temporary.

Abstract Models

What Is an Abstract Model?

- An **abstract model** is a reusable mixin.
- It inherits from **`models.AbstractModel`**.
- It does **not** create its own database table.

Method Syntax

```
class SchoolMailMixin(models.AbstractModel):
    _name = 'school.mail.mixin'
    _description = 'School Mail Mixin'
```

Why Use Abstract Models?

- Share fields and methods across many models.
- Avoid copy-paste logic.
- Classic Odoo examples:
  - `mail.thread`
  - `mail.activity.mixin`
  - `portal.mixin`

Creating and Using a Mixin

```python
class ArchiveMixin(models.AbstractModel):
    _name = 'school.archive.mixin'
    _description = 'Archive Mixin'

    active = fields.Boolean(default=True)

    def action_archive(self):
        self.write({'active': False})

class Student(models.Model):
    _name = 'student.student'
    _inherit = ['school.archive.mixin']
```

- `student.student` receives `active` and `action_archive`.
- No table is created for `school.archive.mixin`.

AbstractModel vs Model vs TransientModel

- `Model`: real persistent business records + table.
- `TransientModel`: temporary wizard records + temporary table.
- `AbstractModel`: reusable definition only, no own table.

```
1 models.Model
2 models.TransientModel
3 models.AbstractModel
```

Abstract Model Best Practices

- Give mixins clear names ending with `mixin` when useful.
- Keep them focused (mailing, archiving, rating, etc.).
- Inherit them with `_inherit = ['my.mixin', ...]`.
- Do not expect to `search()` on an abstract model itself.

Python and SQL Constraints

What Are Constraints?

- Constraints protect data integrity.
- Two main families in Odoo:
  1. **Python constraints** (`@api.constrains`)
  2. **SQL constraints** (`_sql_constraints`)
- Use Python for flexible business rules.
- Use SQL for strong database-level guarantees.

Python Constraints

Method Syntax

```python
from odoo.exceptions import ValidationError

@api.constrains('age')
def _check_age(self):
    for student in self:
        if student.age < 5:
            raise ValidationError("Student age must be at least 5.")
```

- Runs on create/write when listed fields change.
- Can read other fields and related values.

Multi-Field Python Constraint

```python
@api.constrains('start_date', 'end_date')
def _check_period(self):
  for rec in self:
    if rec.start_date and rec.end_date and rec.end_date < rec.start_date:
      raise ValidationError("End date must be after start date.")
```

- Include every field that should trigger validation.
- Raise `ValidationError`, not a generic exception.

SQL Constraints

Method Syntax

```
_sql_constraints = [
  ('name_unique', 'unique(name)', 'Student name must be unique.'),
  ('age_positive', 'CHECK(age >= 0)', 'Age must be positive.'),
]
```

- Each tuple is: `(name, sql_definition, error_message)`.
- Enforced by PostgreSQL.
- Excellent for uniqueness and simple CHECK rules.

Python vs SQL Constraints

- **Python**
  - Flexible logic
  - Better dynamic messages
  - Can use related fields / ORM searches
- **SQL**
  - Strongest guarantee
  - Faster low-level checks
  - Limited to SQL expressions

Practical Example Together

```python
class Student(models.Model):
    _name = 'student.student'

    email = fields.Char()
    age = fields.Integer()

    _sql_constraints = [
        ('email_unique', 'unique(email)', 'Email must be unique.'),
    ]

    @api.constrains('age')
    def _check_age(self):
        for student in self:
            if student.age and student.age > 100:
                raise ValidationError("Age seems invalid.")
```

Extension and Delegation Inheritance

Two Inheritance Styles

- Odoo model inheritance has two major styles:
  1. **Extension inheritance** with `_inherit`
  2. **Delegation inheritance** with `_inherits`
- They solve different problems.
- Understanding both is essential for clean module design.

Extension Inheritance (_inherit)

- Extends an existing model in place.
- Adds fields/methods to the same model/table.

Method Syntax

```
class Student(models.Model):
    _inherit = 'student.student'

    blood_group = fields.Char()
```

- No new model name is required when you only extend.

Classical Extension Patterns

```python
# 1) Extend existing model
class ResPartner(models.Model):
    _inherit = 'res.partner'
    is_teacher = fields.Boolean()

# 2) Prototype inheritance: new model based on another
class StudentAlumni(models.Model):
    _name = 'student.alumni'
    _inherit = 'student.student'
```

- `_inherit` alone extends the original model.
- `_name` + `_inherit` creates a new model copying the parent definition.

Delegation Inheritance (_inherits)

- Creates a new model that **delegates** fields to a linked parent record.
- Classic example: `res.users` delegates to `res.partner`.

Method Syntax

```
class StudentProfile(models.Model):
    _name = 'student.profile'
    _inherits = {'res.partner': 'partner_id'}

    partner_id = fields.Many2one(
        'res.partner', required=True, ondelete='cascade')
    admission_no = fields.Char()
```

How Delegation Works

```python
profile = self.env['student.profile'].create({
    'name': 'Ahmed Muhumed',        # goes to res.partner
    'email': 'ahmed@example.com',   # goes to res.partner
    'admission_no': 'A-001',        # local field
})
print(profile.name)    # delegated field
print(profile.partner_id)
```

- Delegated fields are readable/writable through the child model.
- Data is stored in the parent table.

Extension vs Delegation

- Use **extension** when you want to customize an existing model.
- Use **delegation** when you want a new model that reuses another model's data compositionally.

```
1 _inherit = 'student.student'                # extend
2 _inherits = {'res.partner': 'partner_id'} # delegate
```

- Do not confuse `_inherit` with `_inherits`.

Inheritance Best Practices

- Prefer extension for small feature additions.
- Call `super()` when overriding methods.
- Keep delegated `Many2one` required with a clear `ondelete`.
- Avoid deep inheritance chains that are hard to debug.

XPath and Extend Views

What Is View Inheritance?

- View inheritance lets you modify an existing view without rewriting it.
- You create a new `ir.ui.view` with:
  - `inherit_id` pointing to the parent view
  - XPath / `<field name="...">` instructions describing changes
- This is the standard way to customize Odoo screens.

Basic Inherited View Structure

```
<record id="view_student_form_inherit" model="ir.ui.view">
  <field name="name">student.student.form.inherit</field>
  <field name="model">student.student</field>
  <field name="inherit_id" ref="school_manager.view_student_form"/>
  <field name="arch" type="xml">
  <!-- modification instructions here -->
  </field>
</record>
```

Shortcut: Inherit by Field Name

```
<field name="email" position="after">
  <field name="blood_group"/>
</field>
```

- This finds the field named `email` and applies a position.
- Common positions:
  - `after`
  - `before`
  - `inside`
  - `replace`
  - `attributes`

XPath Inheritance

Method Syntax

```
<xpath expr="//field[@name='age']" position="after">
  <field name="birth_date"/>
</xpath>
```

- `expr` is an XPath expression into the parent architecture.
- More powerful when the target is not a simple field.

Useful XPath Examples

```
<!-- Add a page inside a notebook -->
<xpath expr="//notebook" position="inside">
  <page string="Medical">
  <field name="blood_group"/>
  </page>
</xpath>

<!-- Replace a field -->
<xpath expr="//field[@name='notes']" position="replace">
  <field name="internal_notes"/>
</xpath>

<!-- Change attributes -->
<xpath expr="//field[@name='email']" position="attributes">
  <attribute name="required">1</attribute>
</xpath>
```

Positions Explained

- `before`: insert before the matched node
- `after`: insert after the matched node
- `inside`: insert as a child of the matched node
- `replace`: replace the matched node completely
- `attributes`: modify XML attributes of the matched node

View Extension Tips

- Target stable nodes (named fields, named groups/pages).
- Prefer precise XPath expressions.
- Do not replace large view chunks unless necessary.
- Test after module upgrade because parent views may change.

```
<xpath expr="//page[@name='info']" position="inside">
  <group>
  <field name="guardian_phone"/>
  </group>
</xpath>
```

Smart / Stats Buttons

What Are Smart Buttons?

- Smart buttons (stat buttons) appear at the top of a form view.
- They show a KPI / count and open related records when clicked.
- Excellent for navigation and dashboard-like form headers.

Stat Button in the Form View

```
<div class="oe_button_box" name="button_box">
  <button class="oe_stat_button"
      type="object"
      name="action_view_attendances"
      icon="fa-calendar-check-o">
  <field name="attendance_count" widget="statinfo" string="Attendances"/>
  </button>
</div>
```

- Place them inside `oe_button_box`.
- Use `statinfo` widget for the count/value.

Computed Count Field

```
attendance_count = fields.Integer(compute='_compute_attendance_count')

def _compute_attendance_count(self):
  Attendance = self.env['student.attendance']
  for student in self:
    student.attendance_count = Attendance.search_count([
      ('student_id', '=', student.id),
    ])
```

- The button label usually comes from this computed field.

Button Action Method

```python
def action_view_attendances(self):
    self.ensure_one()
    return {
        'type': 'ir.actions.act_window',
        'name': 'Attendances',
        'res_model': 'student.attendance',
        'view_mode': 'list,form',
        'domain': [('student_id', '=', self.id)],
        'context': {'default_student_id': self.id},
    }
```

- The action opens related records filtered to the current student.
- Context can set defaults for creating new related rows.

Adding a Smart Button by Inheritance

```
<xpath expr="//div[@name='button_box']" position="inside">
  <button class="oe_stat_button" type="object"
      name="action_view_attendances"
      icon="fa-calendar-check-o">
  <field name="attendance_count" widget="statinfo" string="Attendances"/>
  </button>
</xpath>
```

Smart Button Best Practices

- One button = one related document type / KPI.
- Keep count compute methods efficient.
- Use clear icons and short labels.
- Always return an action with a proper domain.
- Call `ensure_one()` in the button method.

Advanced Views

Beyond Form and List

- Form and list/tree are only the beginning.
- Advanced views help users analyze and organize records visually.
- In this section:
  - Kanban
  - Pivot
  - Graph
  - Calendar
  - Activity
  - Gantt
  - Search view reminders for all of them

Declaring Multiple View Modes

```
<record id="action_student" model="ir.actions.act_window">
  <field name="name">Students</field>
  <field name="res_model">student.student</field>
  <field name="view_mode">kanban,list,form,pivot,graph,calendar,activity</field>
</record>
```

- The order in `view_mode` controls the switcher order.
- Each mode needs a corresponding view architecture.

Kanban View

- Kanban shows records as cards, often grouped in columns.

```
<kanban default_group_by="classroom_id" class="o_kanban_small_column">
  <field name="name"/>
  <field name="classroom_id"/>
  <field name="age"/>
  <templates>
  <t t-name="card">
    <div class="fw-bold"><field name="name"/></div>
    <div>Age: <field name="age"/></div>
  </t>
  </templates>
</kanban>
```

- Great for pipelines, classrooms, stages, and boards.

Kanban Tips

- Use `default_group_by` for column grouping.
- Keep cards compact: title + a few key fields.
- Include fields used in the template as `<field/>` declarations.
- In Odoo 17, card templates often use `t-name="card"`.

Pivot View

- Pivot is for interactive spreadsheet-like analysis.

```
<pivot string="Student Analysis">
  <field name="classroom_id" type="row"/>
  <field name="gender" type="col"/>
  <field name="age" type="measure"/>
</pivot>
```

- `type="row" / col"` define axes.
- `type="measure"` defines aggregated values.

Graph View

- Graph shows bar/line/pie charts.

```
<graph string="Students by Classroom" type="bar">
  <field name="classroom_id"/>
  <field name="age" type="measure"/>
</graph>
```

- Useful for dashboards and quick comparisons.
- Common types: `bar`, `line`, `pie`.

Calendar View

- Calendar places records on dates.

```
<calendar string="Student Events"
    date_start="start_datetime"
    date_stop="end_datetime"
    color="classroom_id"
    mode="month">
  <field name="name"/>
  <field name="classroom_id"/>
</calendar>
```

- Needs at least a start datetime/date field.

Activity View

- Activity view focuses on scheduled activities and follow-ups.

```
<activity string="Student Activities">
  <field name="name"/>
  <field name="activity_state"/>
  <field name="activity_ids"/>
</activity>
```

- Usually used with `mail.activity.mixin`.
- Excellent for CRM-like follow-up workflows.

Gantt View

- Gantt shows time ranges on a timeline (Enterprise in many Odoo setups).

```
<gantt string="Classroom Schedule"
    date_start="start_datetime"
    date_stop="end_datetime"
    default_group_by="classroom_id"
    color="teacher_id">
  <field name="name"/>
</gantt>
```

- Ideal for planning, allocation, and scheduling.

Search View Support for Advanced Views

```
<search>
  <field name="name"/>
  <field name="classroom_id"/>
  <filter string="Adults" name="adults" domain="[('age','&gt;=',18)]"/>
  <group>
  <filter string="Classroom" name="group_classroom"
      context="{'group_by': 'classroom_id'}"/>
  <filter string="Gender" name="group_gender"
      context="{'group_by': 'gender'}"/>
  </group>
</search>
```

- Group-by filters are especially important for pivot/graph/kanban.

Choosing the Right Advanced View

- **Kanban**: boards / stages / cards
- **Pivot**: analytical tables
- **Graph**: charts and trends
- **Calendar**: date-based events
- **Activity**: follow-up management
- **Gantt**: schedules and allocations

Navigation and Actions

How Users Reach Your Views

- Menus and window actions connect models to screens.
- A typical chain is:
  1. Menu item
  2. Window action (`ir.actions.act_window`)
  3. View modes + views
- Smart buttons and wizards also return actions for navigation.

Window Action Recap

```
<record id="action_student" model="ir.actions.act_window">
  <field name="name">Students</field>
  <field name="res_model">student.student</field>
  <field name="view_mode">list,kanban,form,graph,pivot</field>
  <field name="context">{}</field>
  <field name="domain">[('active', '=', True)]</field>
</record>
```

Menus

```
<menuitem id="menu_school_root" name="School"/>
<menuitem id="menu_students"
    name="Students"
    parent="menu_school_root"
    action="action_student"
    sequence="10"/>
```

- `parent` builds the menu tree.
- `action` opens the window action.

Navigation from Python

```python
def action_open_classroom(self):
    self.ensure_one()
    return {
        'type': 'ir.actions.act_window',
        'name': 'Classroom',
        'res_model': 'school.classroom',
        'view_mode': 'form',
        'res_id': self.classroom_id.id,
        'target': 'current',
    }
```

- `target`: `current`, `new`, `fullscreen`, etc.
- Buttons, smart buttons, and wizards all use this pattern.

Navigation Best Practices

- Keep menu labels clear and short.
- Put the most common view mode first.
- Use domains/context in actions instead of hard-coding filters in every method.
- Reuse actions from smart buttons and wizards when possible.

Summary

Chapter Four Summary

- **Transient models** power wizard forms and temporary processes.
- **Abstract models** share reusable mixin logic.
- **Python + SQL constraints** protect data quality.
- **_inherit** extends models; **_inherits** delegates to a parent record.
- **XPath / view inheritance** customizes screens cleanly.
- **Smart buttons** expose KPIs and related documents.
- **Advanced views** (Kanban, Pivot, Graph, Calendar, Activity, Gantt) unlock richer UX.

Practical Build Order

1. Create/extend the model and constraints
2. Add form/list views
3. Inherit views with XPath where needed
4. Add smart buttons and actions
5. Add advanced views and search group-bys
6. Add wizards for multi-step processes

Checklist

- Wizard? → `TransientModel` + form + `target='new'`
- Shared logic? → `AbstractModel` mixin
- Validation? → `@api.constrains` and/or `_sql_constraints`
- Extend model/view? → `_inherit` / XPath
- Compose partner-like data? → `_inherits`
- KPI shortcut? → smart button + action
- Analysis/schedule UI? → pivot/graph/calendar/gantt/kanban/activity